

Hadoop与Spark

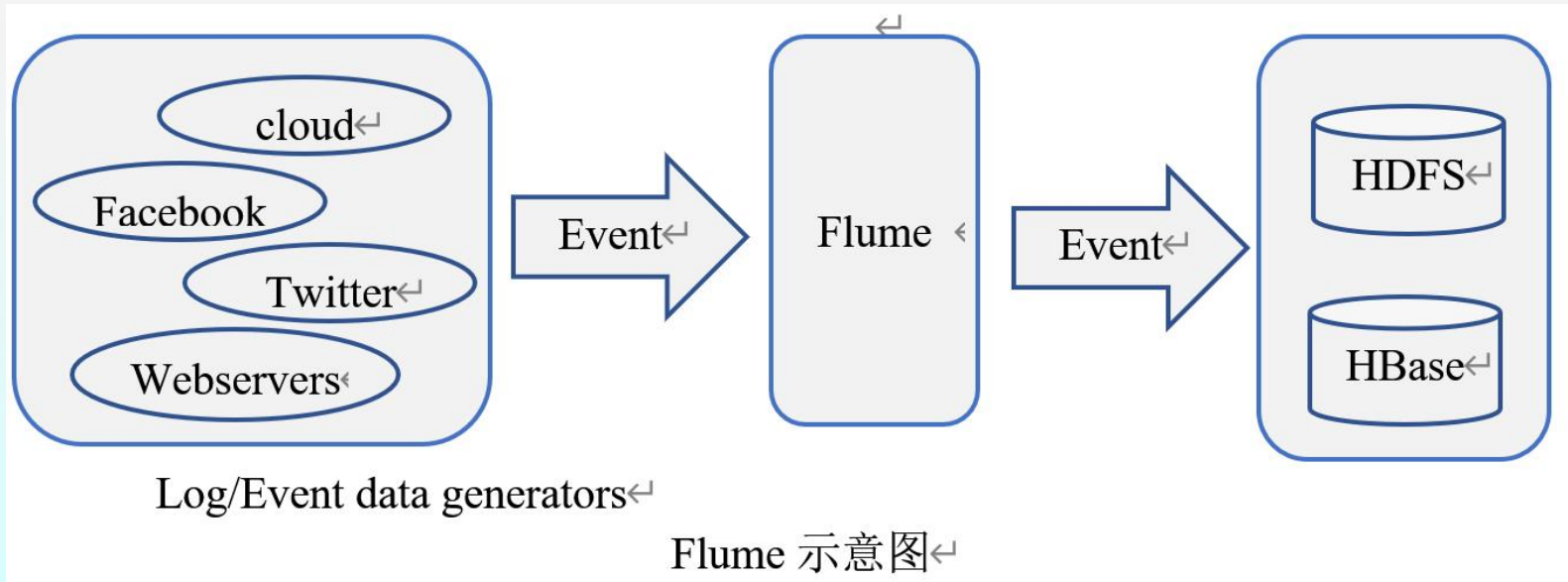
Flume



Flume

- ◆ Flume简介

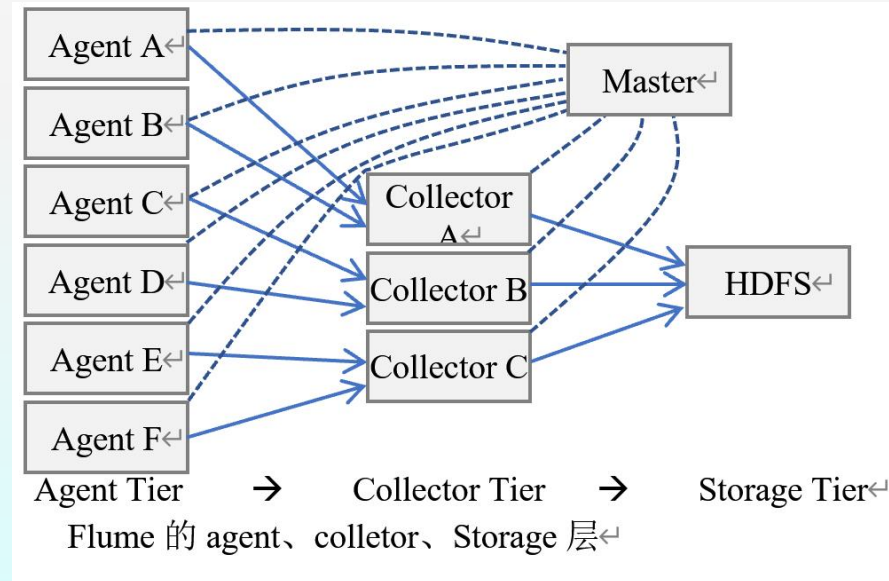
- ◆ Flume是由Cloudera公司研发的分布式日志收集系统，于2009年捐赠了Apache软件基金会，为Hadoop生态系统的相关工具之一
- ◆ Flume是收集日志、事件等数据资源，并将这些数量庞大的数据，从各项数据资源集中起来存储的工具



Flume

◆ Flume简介

- ◆ Flume的组件包括Master、Agent、Collector、Storage等
- ◆ Agent用于采集数据，是Flume中产生数据流的地方
- ◆ Agent将产生的数据流传输到Collector
- ◆ 对应的，Collector用于对数据进行聚合，往往会产生另一个流



Flume

- ◆ Flume的可靠性、可扩展性、可管理性和可扩充性

- ◆ 可靠性(Reliability)

- ◆ Flume提供3种数据可靠性选项，包括End-to-End、Store on Failure和Best Effort。其中End-to-End使用了磁盘日志和接收端Ack的方式，保证Flume接受到的数据会最终到达目的地。Store on Failure在目的地不可用的时候，数据会保持在本地硬盘。和End-to-End不同的是，如果是进程出现问题，Store on Failure可能会丢失部分数据。Best Effort不做任何QoS保证

- ◆ 可扩展性(Scalability)

- ◆ Flume的组件Collector、Master和Storage Tier都是可伸缩的。需要注意的是，Flume对事件的处理不需要带状态，它的Scalability可以很容易实现

- ◆ 可管理性(Manageability)

- ◆ Flume利用ZooKeeper和Gossip协议，保证配置数据的一致性、高可用。同时，Flume使用多个Master，保证Master可以管理大量的节点

- ◆ 可扩充性(Extensibility)

- ◆ 用户可以基于Java为Flume添加各种新的功能
- ◆ 如通过实现Source的子类，用户可以实现自己的数据接入方式。通过实现Sink的子类，用户可以将数据写往特定目标。通过Sink Decorator，用户可以对数据进行一定的预处理

Flume

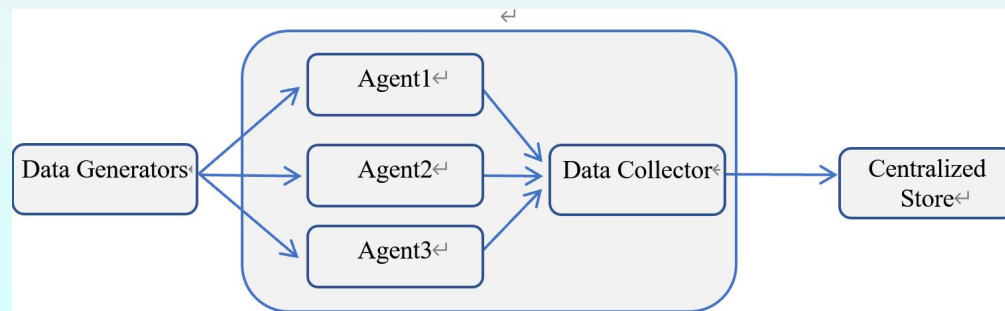
◆ Flume的主要特点

- ◆ (1) 高效率的将多个网站服务器收集的日志信息存入HDFS/HBase;
- ◆ (2) 支持多路径流量, 多管道接入流量, 多管道接出流量, 上下文路由等;
- ◆ (3) 可以水平扩展;
- ◆ (4) Flume的管道是基于事务的, 保证了数据在传送和接收时的一致性;
- ◆ (5) Flume是可靠的、高度容错的、可升级的、易管理的, 并且可定制的;
- ◆ (6) Flume可以收集社交网络站点事件数据, 比如Facebook、Twitter、电商网站如亚马逊等

Flume

◆ Flume的系统架构如图

- ◆ 数据发生器(数据源, 如Facebook、Twitter)产生的数据, 被运行在数据发生器所在服务器上的Agent所收集
- ◆ 之后, 数据收集器(Collector)从各个Agent上汇集数据, 并且将数据存储到某个中心库, 比如HDFS或者HBase中
- ◆ Flume的核心功能, 是把数据从数据源(Source)收集过来, 再将收集到的数据送到指定的目的地(Sink)
- ◆ 为了保证输送的过程能够成功, 在送到目的地之前, 先将数据缓存起来(Channel); Chanel作为一个数据缓冲区临时存放这些数据, 待数据真正到达目的地(Sink)后, 例如HDFS等, Flume再删除自己缓存的数据, 这种机制保证了数据传输的可靠性与安全性
- ◆ 在整个数据的传输的过程中, 流动的是Event, 即事务保证是在Event级别进行的

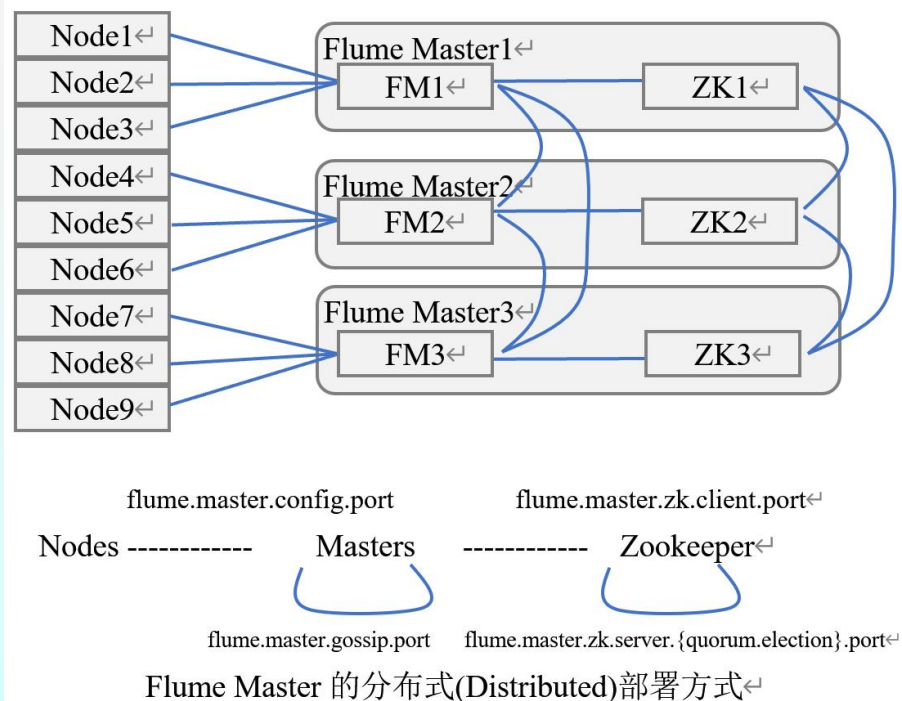


Flume 的系统架构

Flume

◆ Flume的扩展性和可靠性

- ◆ 为了保证可扩展性，Flume采用了多Master的方式
- ◆ 为了保证配置数据的一致性，Flume引入了Zookeeper，用于保存配置数据，Zookeeper本身可保证配置数据的一致性和高可用
- ◆ 另外，在配置数据发生变化时，Zookeeper可以通知Flume Master节点
- ◆ Flume Master间使用Gossip协议同步数据



Flume

- ◆ **Flume OG**: Flume original generation 即 Flume 0.9.x版本
- ◆ **Flume NG**: Flume next generation , 即Flume 1.x版本
 - ◆ 对于 Flume NG , 它摒弃了Master和zookeeper, collector也没有了, web配置台也没有了, 只剩下source, sink和channel
 - ◆ 此时一个Agent的概念包括source, channel和sink, 完全由一个分布式系统变成了传输工具
 - ◆ 不同机器之间的数据传输不再是OG那样由agent->collector, 而是由一个Agent端的sink流向另一个agent的source

Flume

◆ Flume事件

- ◆ Flume事件(Event)是Flume内部数据传输的最基本单元
- ◆ 它由一个装载数据的字节数组, 和一个可选头部构成
- ◆ 典型的Flume 事件, 具有如下图的结构
- ◆ Event将传输的数据进行封装, 如果是文本文件, 通常是一行记录。同时, Event也是事务处理的基本单位



Flume Event 的结构

Flume



Flume Agent 的结构

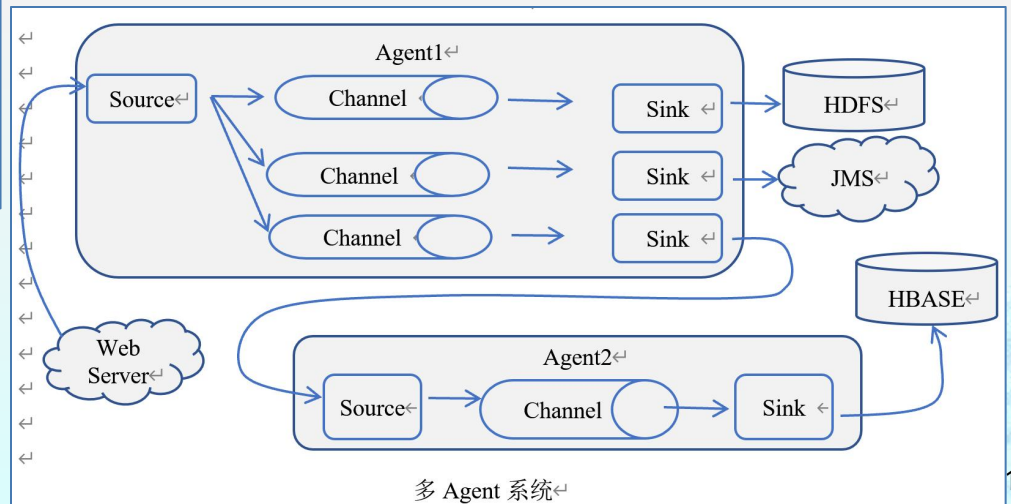
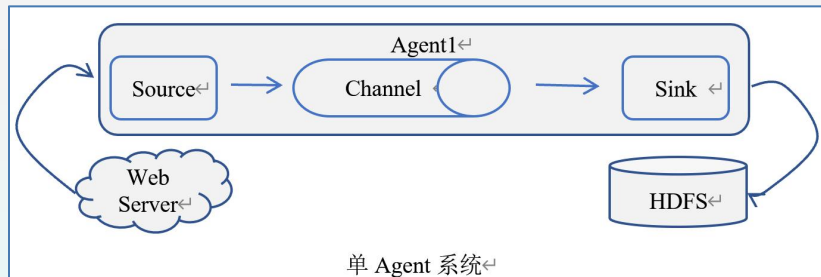
◆ Flume Agent

- ◆ Agent是一共独立的守护进程(JVM)，它从客户端那里接收数据，或者从其它的Agent那里接收数据，然后迅速地将获取的数据传给下一个目的节点(Sink)、或者Agent
- ◆ Agent由Source、Channel、Sink三个组件组成
- ◆ Source从数据发生器接收数据，并将接收的数据以Event格式传递给一个或者多个通道Channel。Flume提供多种数据接收的方式，包括Avro、Thrift等
- ◆ Channel是一种暂时的存储容器，它将从Source处接收到的Event格式的数据缓存起来，直到它们被Sink消费掉。它在Source和Sink间起着桥梁的作用Channel的处理是一个完整的事务，这一点保证了数据在收发的时候的一致性
 - ◆ Channel可以和任意数量的Source和Sink链接。主要的Channel类型有: JDBC Channel、File System Channel、memory Channel等
- ◆ 目的地可能是另一个Sink，也可能是HDFS或者HBase。
 - ◆ Sink将数据存储在集中存储器比如HBase和HDFS。它从Channel消费数据(Event)并将其传递到目的地

Flume

◆ Flume Agent的级联

- ◆ Flume可以前后相继，形成级联结构，比如Sink可以将数据写到下一个Agent的Source中，这样的话就可以连成串，可以整体处理了。Flume提供了大量内置的Source、Channel和Sink类型。不同类型的Source、Channel和Sink可以自由组合。组合方式基于用户设置的配置文件，非常灵活



Flume

- ◆ **Flume如何 保证 数据不丢失?**
- ◆ (1) flume使用两个独立的事务分别负责从source到channel以及从channel到sink的事件传递,
 - ◆ source向channel写数据时, 会先将数据写入临时缓冲区putlist, 然后检查channel是否可以写入, 如果写入成功会更新偏移量, 如果不成功就回滚数据;
 - ◆ channel到sink的事件传递也会先将数据写入临时缓冲区takelist, 如果数据发送成功就更新偏移量, 如果发送失败就回滚数据
- ◆ (2) Channel 使用的是 File Channel
 - ◆ 数据存储在磁盘, 宕机数据可以保存; 但是传输速率慢
 - ◆ 这个能够保证数据不丢失

pause



Flume

- ◆ **Flume**的安装、配置和运行
- ◆ 我们一般下载**Flume**的二进制发行版，解压缩，完成安装

```
cd /opt/linuxsir  
ls apache-flume-1.7.0-bin.tar.gz  
  
tar -zxvf apache-flume-1.7.0-bin.tar.gz  
mv apache-flume-1.7.0-bin apache-flume
```

Flume

- ◆ Flume的安装、配置和运行
- ◆ 从模板拷贝 **flume-env.sh** 配置文件

```
cd apache-flume/conf  
cp flume-env.sh.template flume-env.sh
```

在文件里面设定JAVA_HOME。
`export JAVA_HOME=/opt/linuxsir/java/jdk`

pause



Flume

◆ Flume的安装、配置和运行

◆ 实例1

- ◆ 编辑conf/flume.conf文件，内容如下。通过这个配置文件对Agent进行配置，这里只有一个Agent，即agent1

```
# Define a memory channel called ch1 on agent1
agent1.channels.ch1.type = memory

# Define an Avro source called avro-source1 on agent1 and tell it
# to bind to 0.0.0.0:41414. Connect it to channel ch1
agent1.sources.avro-source1.channels = ch1
agent1.sources.avro-source1.type = avro
agent1.sources.avro-source1.bind = 0.0.0.0
agent1.sources.avro-source1.port = 41414

# Define a logger sink that simply logs all events it receives
# and connect it to the other end of the same channel
agent1.sinks.log-sink1.channel = ch1
agent1.sinks.log-sink1.type = logger

# Finally, now that we've defined all of our components, tell
# agent1 which ones we want to activate
agent1.channels = ch1
agent1.sources = avro-source1
agent1.sinks = log-sink1
```

Flume

- ◆ **Flume的安装、配置和运行**
- ◆ **实例1**

打开一个终端窗口，通过如下命令，启动Flume Server，并启动agent1。

```
cd /opt/linuxsir  
cd apache-flume  
bin/flume-ng agent --conf ./conf/ -f ./conf/flume.conf -Dflume.root.logger=DEBUG,console -n agent1
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例1**

- ◆ 打开另外一个终端窗口，通过如下命令，运行一个Avro 客户端程序，它读取文件，输送到某个主机的Avro Source进程监听的端口上，这里Avro Source运行在本机上。

- ◆ Avro 客户端程序把文件的每一行当作一个事件。Avro客户端程序读取的是/etc/passwd文件。由于agent1的Sink是一个Logger类型的Sink，我们应该可以在第一个终端窗口上观察到相应的输出，也就是/etc/passwd文件的一行行内容

```
cd /opt/linuxsir  
cd apache-flume  
bin/flume-ng avro-client --conf ./conf/ -H localhost -p 41414 -F /etc/passwd -  
Dflume.root.logger=DEBUG,console
```

pause



Flume

◆ Flume的安装、配置和运行

◆ 实例2

◆ 使用netcat的一个Flume实例

◆ 在/opt/linuxsir/apache-flume/conf目录下，创建example.conf，内容如下

```
# Name the components on this agent
a1.sources = r1
a1.sinks = k1
a1.channels = c1

# Describe/configure the source
a1.sources.r1.type = netcat
a1.sources.r1.bind = localhost
a1.sources.r1.port = 44444

# Describe the sink
a1.sinks.k1.type = logger

# Use a channel which buffers events in memory
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

# Bind the source and sink to the channel
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

Flume

- ◆ **Flume的安装、配置和运行**
 - ◆ **实例2**
 - ◆ 使用netcat的一个Flume实例

打开一个终端窗口，通过如下命令，启动Flume Server，并启动a1。

```
cd /opt/linuxsir  
cd apache-flume  
bin/flume-ng agent --conf ./conf/ -f ./conf/example.conf -  
Dflume.root.logger=DEBUG,console -n a1
```

Flume

- ◆ **Flume的安装、配置和运行**
 - ◆ **实例2**
 - ◆ 使用netcat的一个Flume实例

打开另一个终端，连接NetCat。

```
nc localhost 44444
```

并且输入一些内容试试看，这时候应该在第一个终端窗口看到输入的内容。

```
Hello{回车}
```

```
World{回车}
```

pause



Flume

◆ Flume的安装、配置和运行

◆ 实例3

◆ Flume HBase Sink实例

◆ 在/opt/linuxsir/apache-flume/conf目录下，创建hbase_sink.conf，内容如下

```
# Name the components on this agent
Agent5.sources = spooldir-source
Agent5.channels = spillmem-channel
Agent5.sinks = hbase-sink

# Describe/configure Source
Agent5.sources.spooldir-source.type = spooldir
Agent5.sources.spooldir-source.spoolDir = /opt/linuxsir/apache-flume/spooldir
Agent5.sources.spooldir-source.fileHeader = false

# Describe the sink
Agent5.sinks.hbase-sink.type = hbase
Agent5.sinks.hbase-sink.table = test_table
Agent5.sinks.hbase-sink.columnFamily = test_cf

# Use a channel which buffers events in file
Agent5.channels.spillmem-channel.type = SPILLABLEMEMORY
Agent5.channels.spillmem-channel.memoryCapacity = 10000
Agent5.channels.spillmem-channel.overflowCapacity = 1000000
Agent5.channels.spillmem-channel.byteCapacity = 80000
Agent5.channels.spillmem-channel.checkpointDir = /var/log/flume/checkpoint/
Agent5.channels.spillmem-channel.dataDirs = /var/log/flume/data/

# Bind the source and sink to the channel
Agent5.sources.spooldir-source.channels = spillmem-channel
Agent5.sinks.hbase-sink.channel = spillmem-channel
```

Flume

◆ Flume的安装、配置和运行

◆ 实例3

◆ Flume HBase Sink实例

启动HDFS、YARN、HBase, 然后启动HBase Shell。

```
rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
clear
```

```
cd /opt/linuxsir/hadoop/sbin
```

```
./start-dfs.sh
```

```
./start-yarn.sh
```

```
cd /opt/linuxsir/hbase
```

```
./bin/start-hbase.sh
```

```
clear
```

```
jps
```

```
ssh root@192.168.31.130 jps
```

```
ssh root@192.168.31.131 jps
```

```
cd /opt/linuxsir/hbase
```

```
./bin/hbase shell
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例3**

- ◆ **Flume HBase Sink实例**

在HBase Shell里面建立表格test_table, 包含一个Column Family即test_cf。
create 'test_table', 'test_cf'

在终端上

创建Spool目录, 并且进行授权。

```
mkdir /opt/linuxsir/apache-flume/spooldir
```

```
chmod a+rw /opt/linuxsir/apache-flume/spooldir
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例3**

- ◆ **Flume HBase Sink实例**

打开一个终端窗口，通过如下命令，启动Flume Server，并启动Agent5。

```
cd /opt/linuxsir
```

```
cd apache-flume
```

```
bin/flume-ng agent --conf ./conf/ -f ./conf/hbase_sink.conf -
```

```
Dflume.root.logger=DEBUG,console -n Agent5
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例3**

- ◆ **Flume HBase Sink实例**

打开另一个终端，把一个文本文件拷贝到/opt/linuxsir/apache-flume/spooldir，文件的每一行被当作独立的事件，发送到Flume，并且被插入到HBase。

```
cd /opt/linuxsir/  
cp hive-test.txt /opt/linuxsir/apache-flume/spooldir
```

这时候查看/opt/linuxsir/apache-flume/spooldir目录，内容如下。

```
hive-test.txt.COMPLETED
```

Flume

◆ Flume的安装、配置和运行

◆ 实例3

◆ Flume HBase Sink实例

重新启动HBase Shell。

```
cd /opt/linuxsir/hbase  
./bin/hbase shell
```

查看test_table表格。

```
scan 'test_table'
```

结果如下，发现最后一个空行也差进入了，所以在准备数据的时候，要删掉空行。

```
ROW          COLUMN+CELL  
default0bce3a6c-8162-4 column=test_cf:pCol, timestamp=1596711289189, value=4 hello  
375-816c-b47f2143c403  
default47e860fc-6509-4 column=test_cf:pCol, timestamp=1596711289189, value=3 hbase  
66e-9967-e3b036cd14e8  
defaulta8450648-6960-4 column=test_cf:pCol, timestamp=1596711289189, value=1 hadoop  
f9e-8dec-13f3221a1153  
defaultdc7ae64f-8c98-4 column=test_cf:pCol, timestamp=1596711289189, value=2 hive  
5ee-8a2f-22d088eb63bb  
incRow      column=test_cf:iCol, timestamp=1596711289300, value=\x00\x00\x00\x  
00\x00\x00\x00\x00\x04
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例3**

- ◆ **Flume HBase Sink实例**

退出HBase Shell, 最后停止HBase、YARN、HDFS。

```
cd /opt/linuxsir/hbase
```

```
./bin/stop-hbase.sh
```

```
cd /opt/linuxsir/hadoop/sbin
```

```
./stop-yarn.sh
```

```
./stop-dfs.sh
```

```
jps
```

```
ssh root@192.168.31.130 jps
```

```
ssh root@192.168.31.131 jps
```

pause



Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例4**

- ◆ **Flume Hive Sink实例**

首先建立数据文件。

```
cd /opt/linuxsir
```

```
touch flume_message.data
```

```
echo "1,hive" >> flume_message.data
```

```
echo "2,flume" >> flume_message.data
```

```
echo "3,testmessage" >> flume_message.data
```

```
echo "4,hivesink" >> flume_message.data
```

```
cat flume_message.data
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ 实例4

- ◆ Flume Hive Sink实例

- ◆ 在/opt/linuxsir/apache-flume/conf目录下, 创建hive_sink.conf, 内容如下

```
agent1.sources = source1  
agent1.channels = channel1  
agent1.sinks = sink1
```

```
agent1.sources.source1.type = exec  
agent1.sources.source1.command = cat /opt/linuxsir/flume_message.data  
agent1.sources.source1.channels = channel1
```

Flume

◆ Flume的安装、配置和运行

◆ 实例4

◆ Flume Hive Sink实例

◆ 在/opt/linuxsir/apache-flume/conf目录下，创建hive_sink.conf，内容如下

```
agent1.sinks.sink1.type = hive
agent1.sinks.sink1.channel = channel1
agent1.sinks.sink1.hive.metastore = thrift://192.168.31.129:19083
agent1.sinks.sink1.hive.database = default
agent1.sinks.sink1.hive.table = flume_test
agent1.sinks.sink1.hive.txnsPerBatchAsk = 2
agent1.sinks.sink1.batchSize = 4
agent1.sinks.sink1.useLocalTimeStamp = false
agent1.sinks.sink1.round = true
agent1.sinks.sink1.roundValue = 10
agent1.sinks.sink1.roundUnit = minute
agent1.sinks.sink1.serializer = DELIMITED
agent1.sinks.sink1.serializer.delimiter = ","
agent1.sinks.sink1.serializer.serdeSeparator = ','
agent1.sinks.sink1.serializer.fieldnames = id,message

agent1.channels.channel1.type = FILE
agent1.channels.channel1.transactionCapacity = 1000000
agent1.channels.channel1.checkpointInterval 30000
agent1.channels.channel1.maxFileSize = 2146435071
agent1.channels.channel1.capacity 10000000
```

Flume

◆ Flume的安装、配置和运行

◆ 实例4

◆ Flume Hive Sink实例

启动Hive Shell, 准备建立表格。注意, 首先需要启动HDFS和YARN。

```
rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
clear
```

```
cd /opt/linuxsir/hadoop/sbin
```

```
./start-dfs.sh
```

```
./start-yarn.sh
```

```
clear
```

```
jps
```

```
ssh root@192.168.31.130 jps
```

```
ssh root@192.168.31.131 jps
```

```
cd /opt/linuxsir/hive
```

```
./bin/hive --service metastore -p 19083&
```

\\ 等待metastore启动

```
netstat -lanp | grep 19083
```

```
cd $HIVE_HOME/bin
```

```
./hive
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例4**

- ◆ **Flume Hive Sink实例**

在Hive Shell里运行如下SQL，建立表格flume_test，注意需要设定buckets，格式为ORC，支持transaction。
`create table flume_test(id string, message string) clustered by (message) into 5 buckets STORED AS ORC
TBLPROPERTIES ("transactional"="true");`

\\退出Hive Shell
`quit;`

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例4**

- ◆ Flume Hive Sink实例

- ◆ 修改/opt/linuxsir/hive/conf/hive-site.xml, 对若干配置项, 确认其设置如下。注意, 划掉的配置项不需要

```
<property>
<name>hive.txn.manager</name>
<value>org.apache.hadoop.hive.ql.lockmgr.DbTxnManager</value>
</property>
<property>
<name>hive.compactor.initiator.on</name>
<value>true</value>
</property>
<property>
<name>hive.compactor.worker.threads</name>
<value>5</value>
</property>
```

Flume

- ◆ **Flume的安装、配置和运行**

- ◆ **实例4**

- ◆ Flume Hive Sink实例

- ◆ 修改/opt/linuxsir/hive/conf/hive-site.xml, 对若干配置项, 确认其设置如下。注意, 划掉的配置项不需要

```
<property>
<name>hive.compactor.check.interval</name>
<value>10</value>
</property>
<property>
<name>hive.compactor.delta.num.threshold</name>
<value>2</value>
</property>
<property>
<name>hive.support.concurrency</name>
<value>true</value>
</property>
<property>
<del>name>hive.enforce.bucketing</del>
<del>value>true</del>
</property>
<property>
<name>hive.exec.dynamic.partition.mode</name>
<value>nonstrict</value>
</property>
```

Flume

- ◆ **Flume的安装、配置和运行**
 - ◆ 实例4
 - ◆ Flume Hive Sink实例

拷贝Hive的相关Jar包到Flume的lib目录下。

```
cp /opt/linuxsir/hive/hcatalog/share/hcatalog/* /opt/linuxsir/apache-flume/lib  
cp /opt/linuxsir/hive/lib/* /opt/linuxsir/apache-flume/lib
```

Flume

- ◆ **Flume的安装、配置和运行**
 - ◆ 实例4
 - ◆ Flume Hive Sink实例

打开一个终端窗口，通过如下命令，启动Flume Server，并启动agent1。

```
cd /opt/linuxsir  
cd apache-flume  
bin/flume-ng agent --conf ./conf/ -f ./conf/hive_sink.conf -  
Dflume.root.logger=DEBUG,console -n agent1
```

Flume

- ◆ **Flume的安装、配置和运行**
 - ◆ 实例4
 - ◆ Flume Hive Sink实例

打开另一个终端，启动Hive Shell，执行如下查询，看看数据是否已经插入到表格中。

```
cd $HIVE_HOME/bin  
./hive
```

```
select * from flume_test;  
exit;
```

pause



Flume

- ◆ Flume的安装、配置和运行
 - ◆ 附录
 - ◆ 如何安装netcat

如果没有安装netcat，那么在已经配置Yum源的情况下(参考第二章)，可以使用如下命令安装netcat。
`yum install -y nc`

NetCat是一个非常简单的Unix工具，可以读、写TCP或UDP网络连接(network connection)。

最简单的使用方法，”nc host port”，能建立一个TCP连接，连向指定的主机和端口。接下来，你的从标准输入中输入的任何内容都会被发送到指定的主机，任何通过连接返回来的信息都被显示在你的标准输出上。这个连接会一直持续下去，至到连接两端的程序关闭连接。

pause

